

Lab 1: NIST and Inter-chip HD for 2 PUFs

Summary: The following assignment involves utilizing the NIST test suite to assess the randomness of generated bitstreams for two different PUF devices, the first producing bitstreams of length 5093, and the second producing bitstreams of length 864. Eight out of the fifteen NIST tests were applied separately on both bitstreams, and histograms were rendered for each. The data from the histograms was then compared against the binomial distribution that separately characterized each of the bitstreams.

Author: Yo Momma

Date: 25MAR2024

Course: ECE525 Hardware Oriented Security and Trust

Powered by:

MathJax

1 Data Analysis and Visualization

The [NIST Test Suite](#) was retrieved from the official hosting server, and the `assess` binary was compiled via the included makefile. Two bitstring files were provided and served as candidates to be processed by the `assess` binary. The eight tests and their evaluation objectives are listed below:

1. **Frequency:** The focus of the test is the proportion of zeroes and ones for the entire sequence. The purpose of this test is to determine whether that number of ones and zeros in a sequence are approximately the same as would be expected for a truly random sequence. The test assesses the closeness of the fraction of ones to $\frac{1}{2}$, that is, the number of ones and zeroes in a sequence should be about the same.

2. **Block Frequency:** The focus of the test is the proportion of zeroes and ones within M-bit blocks. The purpose of this test is to determine whether the frequency of ones in an M-bit block is approximately $M/2$.

3. **Cumulative Sums:** The focus of this test is the maximal excursion (from zero) of the random walk defined by the cumulative sum of adjusted (-1, +1) digits in the sequence. The purpose of the test is to determine whether the cumulative sum of the partial sequences occurring in the tested sequence is too large or too small relative to the expected behavior of that cumulative sum for random sequences. This cumulative sum may be considered as a random walk. For a random sequence, the random walk should be near zero. For non-random sequences, the excursions of this random walk away from zero will be too large.

4. **Runs:** The focus of this test is the total number of zero and one runs in the entire sequence, where a run is an uninterrupted sequence of identical bits. A run of length k means that a run consists of exactly k identical bits and is bounded before and after with a bit of the opposite value. The purpose of the runs test is to determine whether the number of runs of ones and zeros of various lengths is as expected for a random sequence. In particular, this test determines whether the oscillation between such substrings is too fast or too slow.

5. **Longest Run of Ones:** The focus of the test is the longest run of ones within M-bit blocks. The purpose of this test is to determine whether the length of the longest run of ones within the tested sequence is consistent with the length of the longest run of ones that would be expected in a random sequence. Note that an irregularity in the expected length of the longest run of ones implies that there is also an irregularity in the expected length of the longest run of zeroes. Long runs of zeroes were not evaluated separately due to a concern about statistical independence among the tests.

6. **Discrete Fourier Transform:** The focus of this test is the peak heights in the discrete Fast Fourier Transform. The purpose of this test is to detect periodic features (i.e., repetitive patterns that are near each other) in the tested sequence that would indicate a deviation from the assumption of randomness.

7. **Approximate Entropy:** The focus of this test is the frequency of each and every overlapping m-bit pattern. The purpose of the test is to compare the frequency of overlapping blocks of two consecutive/adjacent lengths (m and m+1) against the expected result for a random sequence.

8. **Serial:** The focus of this test is the frequency of each and every overlapping m-bit pattern across the entire sequence. The purpose of this test is to determine whether the number of occurrences of the 2m m-bit overlapping patterns is approximately the same as would be expected for a random sequence. The pattern can overlap.

1.1

Bitstream 1: 5093 bits/trial

The first set of bitstrings included a text file containing the resulting bitstreams from 500 consecutive trials, with each bitstream containing 5093 bits of information. The results of the NIST test are provided in the figure below:

Figure 1: Result of Bitstream I Assessment

RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES											
generator is											
C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION
62	54	58	47	53	50	44	40	56	36	0.202268	496/500
77	60	68	52	44	39	40	49	45	34	0.000546	488/500
72	66	68	36	38	45	57	30	46	50	0.000114	495/500
58	54	50	41	51	52	50	36	62	46	0.316052	494/500
41	63	45	54	48	46	61	49	45	48	0.415422	498/500
54	48	57	58	46	51	51	50	43	42	0.809249	496/500
59	52	38	45	33	69	52	50	51	51	0.032923	491/500
499	1	0	0	0	0	0	0	0	0	0.000000	18/500
42	44	65	51	53	52	51	51	52	39	0.408275	492/500
42	55	60	49	57	48	45	52	45	47	0.727851	496/500
STATISTICAL TEST											
											Frequency
											BlockFrequency
											CumulativeSums
											CumulativeSums
											Runs
											LongestRun
											FFT
											ApproximateEntropy
											Serial
											Serial

The minimum pass rate for each statistical test with the exception of the random excursion (variant) test is approximately = 488 for a sample size = 500 binary sequences.

For further guidelines construct a probability table using the MAPLE program provided in the addendum section of the documentation.

1.2

Bitstream 2: 864 bits/trial

The second set of bitstreams was analyzed using the same procedure described in the prior analysis. The results of the NIST test are provided in the figure below:

Figure 2: Result of Bitstream 2 Assessment

RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES

generator is ../RawBitstrings_RT_3_XMR_1.txt

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	STATISTICAL TEST
0	0	2	2	5	4	2	6	3	10	0.000954	34/34	Frequency
0	0	0	0	0	0	0	4	6	24	0.000000 *	34/34	BlockFrequency
0	0	0	0	1	4	2	5	2	28	0.000000 *	34/34	CumulativeSums
0	0	1	1	1	1	2	6	2	28	0.000000 *	34/34	CumulativeSums
8	5	5	1	3	1	2	1	3	5	0.054199	33/34	Runs
23	4	4	1	2	0	0	0	0	0	0.000000 *	23/34	* LongestRun
5	2	7	0	5	0	8	0	4	3	0.002043	33/34	FFT
0	0	0	0	0	0	0	0	0	34	0.000000 *	34/34	ApproximateEntropy
4	4	3	5	2	0	6	3	5	2	0.350485	34/34	Serial
6	3	2	0	7	4	3	4	2	3	0.178278	34/34	Serial

The minimum pass rate for each statistical test with the exception of the random excursion (variant) test is approximately = 31 for a sample size = 34 binary sequences.

For further guidelines construct a probability table using the MAPLE program provided in the addendum section of the documentation.

1.3 Histogram Renders

The plotly.js library was utilized to render the histograms for each set of bitstreams, and are presented in the figures below:

Figure 3: Bitstream 1 Histogram

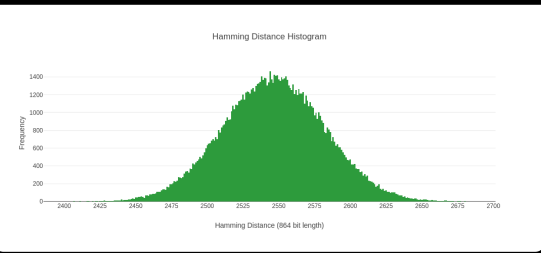
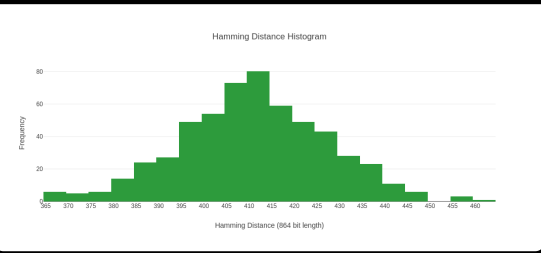


Figure 4: Bitstream 2 Histogram



2 Performance characterization

The performance of the PUFs are taken from the histogram data and compared against the expected binomial distribution for sets of similar sizes. The mean and standard deviation of a binomial distribution are provided below:

Equation 1: Binomial Mean and Standard Deviation

$$\text{Mean}(\mu) = n \times p$$

$$\text{Standard Deviation}(\sigma) = \sqrt{n \times p \times (1 - p)}$$

With the knowledge that there is a 50% probability of a 1 or 0 bit occurring, the mean and standard deviation for Bitstream 1 are $\mu = 5093 \times 0.5 = 2546.5$,

$\sigma = \sqrt{5093 \times 0.5 \times (1 - 0.5)} = 35.683$; the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. In order to calculate the mean and standard deviation relative to the histogram data, the below equation set must be used:

Equation 2: Histogram Mean and Standard Deviation

$$\text{Mean} = \frac{\sum (\text{Bin Center} \times \text{Frequency})}{\sum \text{Frequency}}$$

$$\text{Variance} = \frac{\sum ((\text{Bin Center} - \text{Mean})^2 \times \text{Frequency})}{\sum \text{Frequency}}$$

In order to ensure consistency, the bin centers will be based upon a histogram resolution of 1, which will encompass all the data in the set without conflation. The values calculated for Bitstream 1 evaluate to $\mu_{\text{histogram}} = 2546.411$ and $\sigma_{\text{histogram}} = 35.812$. The values calculated for Bitstream 2 evaluate to $\mu_{\text{histogram}} = 411.034$ and $\sigma_{\text{histogram}} = 16.387$.

3 Results and Discussion

The data report from the NIST assessment and the mathematical evaluation of the performance values present some features that present concerns as to the integrity of the PUF bitstreams. With regard to the NIST test for Bitstream 1, 18 out of the 500 strings indicated features beyond the acceptable threshold for the approximate entropy test, and this may indicate that the PUF bitstream is not as unique or unpredictable as desired. This could potentially weaken the security of cryptographic systems relying on the uniqueness of PUF-generated keys or identifiers. Bitstream 2 had poor performance for a larger proportion of trials, failing to show a high degree of randomness for the block frequency, cumulative sums, longest run, and approximate entropy tests. Note that low p values are indicated with the asterisk (*) and denote low probability of supporting the null hypothesis, which assumes the data is randomly distributed. The mediocre performance of Bitstream 1 and poor performance of Bitstream 2 are corroborated in the mathematical analysis of the histogram values and, thus, their deviance from the expected binomial values. This can be quantified by evaluating the percent error for the mean and standard deviation values obtained from the histogram when compared against the expected values. For Bitstream 1, $\mu_{\text{error}} = -0.003\%$, and $\sigma_{\text{error}} = 0.362\%$, indicating that although the Approximate Entropy test did show a degree of non-randomness, the distribution still appears to adhere closely to the expected binomial distribution, and the PUF may benefit from further investigation as to the failures observed during the Approximate Entropy test. For Bitstream 2, $\mu_{\text{error}} = -4.853\%$, and $\sigma_{\text{error}} = 11.499\%$, indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols.

APPENDIX A

Figure A.1: Code for hammingDistance.js

```
document.getElementById('loadDataBtn').addEventListener('click', () => {
  // Get the selected file from the input
  const fileInput = document.getElementById('fileInput');
  const file = fileInput.files[0];

  if (!file) {
    alert('Please select a file.');
```

```
    return;
  }

  // Read the file using FileReader API
  const reader = new FileReader();
  reader.onload = function(event) {
    const bitstrings = event.target.result.trim().split('\n');

    // Array to store Hamming distances
    const distances = [];

    // Calculate Hamming distance for each pair of bitstrings
    for (let i = 0; i < bitstrings.length - 1; i++) {
      for (let j = i + 1; j < bitstrings.length; j++) {
        const distance = hammingDistance(bitstrings[i], bitstrings[j]);
        distances.push(distance);
      }
    }

    const distancesCount = {}; // Empty JavaScript object to store counts

    // Iterate over the distances array
    distances.forEach(distance => {
      // Check if the property already exists in the object
      if (distancesCount[distance]) {
        // If the property exists, increment its value by one
        distancesCount[distance]++;
      } else {
        // If the property doesn't exist, create a new property with value 1
        distancesCount[distance] = 1;
      }
    });

    //Evaluating the mean
    var histogramTotal = 0;
    var histogramAvg = 0;
    for (const key in distancesCount) {
      histogramTotal += key * distancesCount[key];
    };
    histogramAvg = histogramTotal / distances.length;

    //Evaluating the standard Deviation
    var histogramVar = 0; //A total sum of the variances augmented by their respective weights (frequency of occurrence)
    var histogramVarNorm = 0; //Normalizes the augmented weighted variance against the frequency counts
    var histogramStdDev = 0;

    for (const key in distancesCount) {
      histogramVar += ((key - histogramAvg)**2) * distancesCount[key];
    };
    histogramVarNorm = histogramVar / distances.length;
    histogramStdDev = Math.sqrt(histogramVarNorm);

    // Generate histogram using plotly.js
    const trace = {
      x: distances,
      type: 'histogram',
      marker: {color: '#2e9f40'}
    };
    const layout = {
      title: 'Hamming Distance Histogram',
      xaxis: {
        title: 'Hamming Distance (864 bit length)',
        tickmode: 'linear', // Set tick mode to linear
        tick0: 0, // Start the ticks at 0
        dtick: 25, // Set the tick interval to 1
      },
      yaxis: { title: 'Frequency' },
    };
    Plotly.newPlot('histogram', [trace], layout);
  };

  reader.readAsText(file);
});

function hammingDistance(str1, str2) {
  if (str1.length !== str2.length) {
    throw new Error('Bitstrings must have the same length');
  }
  let distance = 0;
  for (let i = 0; i < str1.length; i++) {
    if (str1[i] !== str2[i]) {
      distance++;
    }
  }
  return distance;
}
```

Indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. This can be rectified by placing the span element in it's own div, which then effectively removes the inheritance of the single block element from the numbering grid feature. For Bitstream 2, $\mu_{\%error} = -4.853\%$, and $\sigma_{\%error} = 11.499\%$, indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms.

Indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. This can be rectified by placing the span element in it's own div, which then effectively removes the inheritance of the single block element from the numbering grid feature. For Bitstream 2, $\mu_{\%error} = -4.853\%$, and $\sigma_{\%error} = 11.499\%$, indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. indicating that the mean is slightly deviant from the expected value, but perhaps more concerning is the large standard deviation percent error, which suggest large variation and much greater likelihood of encountering bitstreams with insufficient randomness to be implemented in cryptographic protocols. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms. the mean and standard deviation values for Bitstream 2 are $\mu = 864 \times 0.5 = 432$ and

$\sigma = \sqrt{864 \times 0.5 \times (1 - 0.5)} = 14.697$. These values will be used as the ideal reference when comparing to the values obtained in the respective histograms.

